l-Diversity for Top-Down Greedy Anonymization

Lara Schwabe, Livia Zylja

SPAI 2026, Exercise 1.2 (Anonymisation), October 2026

1 Introduction

k-anonymity [3] guarantees that every record shares its quasi-identifier (QI) values with at least k-1 other records, so a record cannot be linked to a single row. It says nothing about the sensitive attribute (SA), though. If all records of an equivalence class (EC) have the same SA value, an attacker who knows the QIs learns the sensitive value anyway (homogeneity attack). l-diversity [1] closes this gap by requiring that every EC contains at least l "well-represented" SA values.

We took the Top-Down Greedy (TDG) algorithm from the kaylode k-anonymity toolbox [4] and changed it so that it produces tables that are both k-anonymous and l-diverse. We compared it with the original TDG on the California Housing dataset (cahousing) of the toolbox, with ocean_proximity as SA. This dataset shows the problem very clearly, because the distance to the ocean follows almost directly from longitude and latitude: with the original TDG at k = 10, 687 ECs are homogeneous and they contain 10,192 of the 20,640 districts (49%); at k = 2 it is even 85%.

2 Method

2.1 Top-Down Greedy in the toolbox

TDG [2] starts with the whole table as one partition and splits it recursively. To split a partition it picks two records u and v that are far apart. The distance is the NCP of the generalisation that covers both records, and instead of checking all pairs the toolbox uses a heuristic with three rounds (start at a random record, jump to the farthest one, repeat). Every other record goes to the seed it is closer to. If one side ends up with fewer than k records, the algorithm either moves the nearest records from the other side or merges both sides again, whichever has the lower NCP. Partitions with fewer than 2k records are not split any more, and in the end each partition is generalised to the lowest common ancestor in the hierarchies.

2.2 Our modification

We added the parameters l, the sensitive attribute and the type of l-diversity. A split is only accepted if both halves are k-anonymous and l-diverse. We implemented the three definitions from [1]:

- **distinct l-diversity:** at least l different SA values in the EC,
- **entropy l-diversity:** the entropy of the SA distribution in the EC is at least log(l),
- **recursive (c, l)-diversity:** $r1 < c (rl + rl+1 + ... + rm)$, where ri is the i-th most frequent SA count.

Since TDG splits only by QI distance, one half of a split is often not l-diverse. There are three ways to handle this. Reject: keep the partition as one EC. Retry: draw a new random pair and split again, up to 10 times (this was our first version). Repair, which is what topdown_ldiv uses:

1. Take the half that is not valid (if both are invalid, the split is rejected).
2. Sort the records of the valid half by their NCP distance to the invalid half.
3. Move a record over only if the invalid half still needs records to reach k or the record improves its diversity (new SA value, higher entropy or larger recursive margin), and the valid half stays k-anonymous and l-diverse without it.
4. Stop as soon as the invalid half is valid. If this does not work, or the repaired split has a higher total NCP than not splitting, the partition is kept as one EC.

Since we check at the start that the whole table is l-diverse and only valid partitions are ever created, every EC in the output fulfils k and l. With l = 0 the original code path is used. We also fixed the random seed of the pair heuristic and a global list that was not reset between runs. The method is called with, e.g., python anonymize.py --method=topdown_ldiv --k=10 --l=2 --dataset=cahousing. Unit tests check that every EC has at least k records, is l-diverse and that no SA value was changed.

3 Experimental setup

cahousing has 20,640 districts. The QIs are longitude, latitude, housing_median_age, median_income and median_house_value with the hierarchies from the toolbox. The SA ocean_proximity has only three values (4,953 / 9,136 / 6,551 records), so only l = 2 and 3 are possible, and entropy l = 3 is impossible (the whole table only reaches 2.91). In the toolbox this column is also the classification target, and we kept that. We ran three experiments:

- **B:** k in {2, 3, 5, 10, 20, 50, 100} for the original TDG and for TDG with distinct l = 2, distinct l = 3 and entropy l = 2.
- **C:** k = 2 and 10, distinct l = 2 and 3, with the three strategies reject, retry and repair.
- **R:** recursive (c, l)-diversity with c = 2 and l = 2, 3 at k = 10.

Information loss is measured with NCP, the discernibility metric (DM), the normalised average EC size (CAVG) and the number of ECs. For privacy we count homogeneous ECs and the records in them, and we measure how often an attacker is right who guesses the most frequent SA value of a person's EC ("majority share"). Without knowing any QI this guess is right for 44% of the districts. For classification we used the toolbox pipeline (based on [5]): KNN, SVM and Random Forest are trained on the anonymised training split (80%) to predict ocean_proximity and evaluated with weighted F1 on the anonymised test split. While doing this we found and fixed two bugs in the toolbox evaluation that only show up on cahousing: numeric values with a minus sign or a decimal point (e.g. -122.2) were replaced by 0, and the anonymised rows were matched with the labels by position instead of by ID, so features and labels belonged to different districts. Fully suppressed values (*) are now replaced by the mean of the attribute. KNN uses unscaled features as in the toolbox, which is why it is weak even on the original data.

4 Results

4.1 Privacy

Table 1: Experiment B. Homogeneous ECs of the original TDG and majority share (how often guessing the most frequent SA value of the EC is right) for all methods. TDG with l-diversity has no homogeneous EC in any run.

k	TDG homog. / all ECs	TDG records in homog. ECs	TDG majority share	distinct l=2	distinct l=3	entropy l=2
2	7,356 / 8,554	85%	94%	89%	82%	61%
3	3,841 / 5,015	75%	92%	89%	81%	61%
5	1,778 / 2,740	63%	90%	88%	80%	62%
10	687 / 1,385	49%	88%	87%	81%	61%
20	257 / 738	35%	85%	85%	80%	61%
50	51 / 300	17%	81%	80%	78%	60%
100	13 / 139	9%	77%	76%	73%	59%

The original TDG leaks a lot on this dataset (Table 1). Even at k = 100 there are still 13 homogeneous ECs with 1,776 districts. topdown_ldiv had zero homogeneous ECs and zero violations in every run. However, distinct l-diversity only removes certainty, not the inference: the ECs are often very skewed (in the k = 10, l = 2 run there are ECs with 393 districts of one value and 2 of another, see experiments/cahousing_demo_k10_l2.md), so the majority guess is still right for 89% (l = 2) and 82% (l = 3) of the districts at k = 2, compared with 94% for the original TDG. Only entropy l = 2 really brings it down to about 61%, close to the 44% an attacker gets without any QI.

4.2 Information loss

Figure 1 shows the price. Distinct l = 2 doubles the NCP at small k (0.155 -> 0.319 at k = 2, 0.254 -> 0.351 at k = 10), but the gap gets smaller with larger k (0.566 vs. 0.587 at k = 100). Compared at the same information loss, l-diversity is clearly the better deal: TDG with l = 2 at k = 2 has NCP 0.319 and no homogeneous EC, while the original TDG at k = 20 has a similar NCP (0.322) and still 7,255 districts in homogeneous ECs. Distinct l = 3 costs NCP 0.56 to 0.68 and entropy l = 2 about 0.80, with only 77 ECs at k = 10. Recursive diversity with c = 2 is even stricter: l = 2 gives NCP 0.864, and for l = 3 the whole table already fulfils the condition but no split can keep both halves valid, so everything stays in one EC (NCP 1.0).

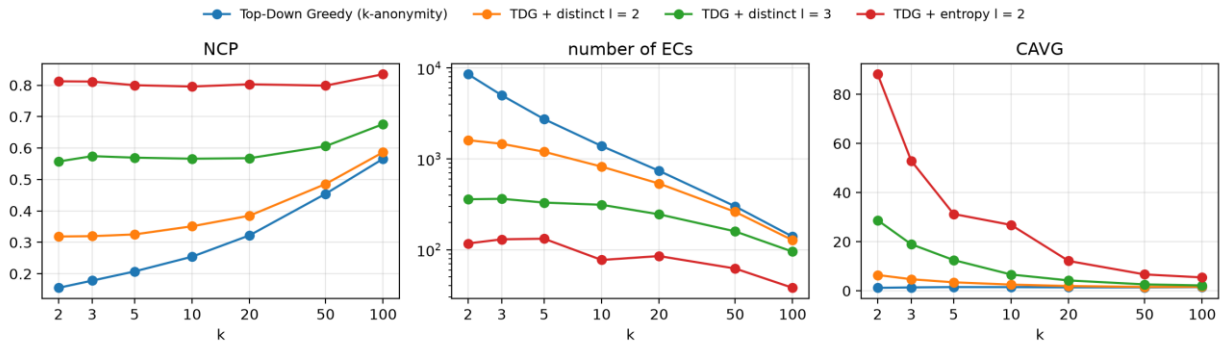


Figure 1: Experiment B. NCP, number of ECs (log scale) and CAVG for different k.

4.3 Reject, retry or repair

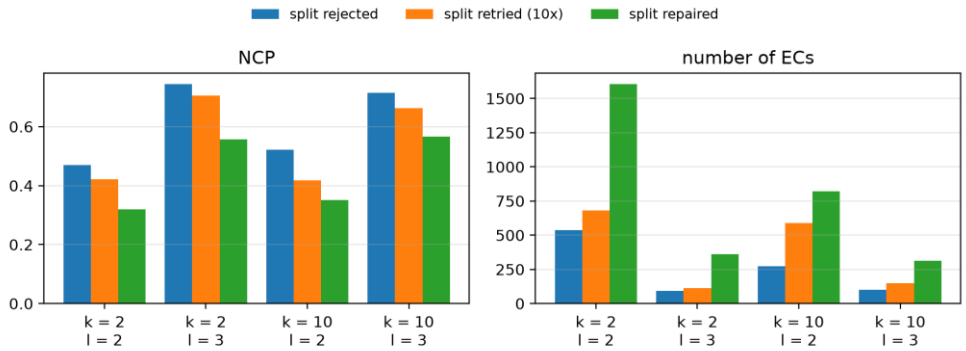


Figure 2: Experiment C. Handling of splits that are not l-diverse (distinct l-diversity).

Repairing the split gives the lowest information loss in all four settings (Figure 2): at k = 2, l = 2 the NCP is 0.469 with reject, 0.421 with retry and 0.319 with repair, at k = 10, l = 3 it is 0.714, 0.663 and 0.566. Retrying helps a bit, but a new random split has the same problem as the first one, because the SA is still not considered. Moving a few records with the missing values fixes exactly that. The repair also keeps the most ECs, which means its ECs are smaller and the majority share is a little higher (e.g. 81% vs. 79% for retry at k = 10, l = 3), but all of them fulfil the same l.

4.4 Classification

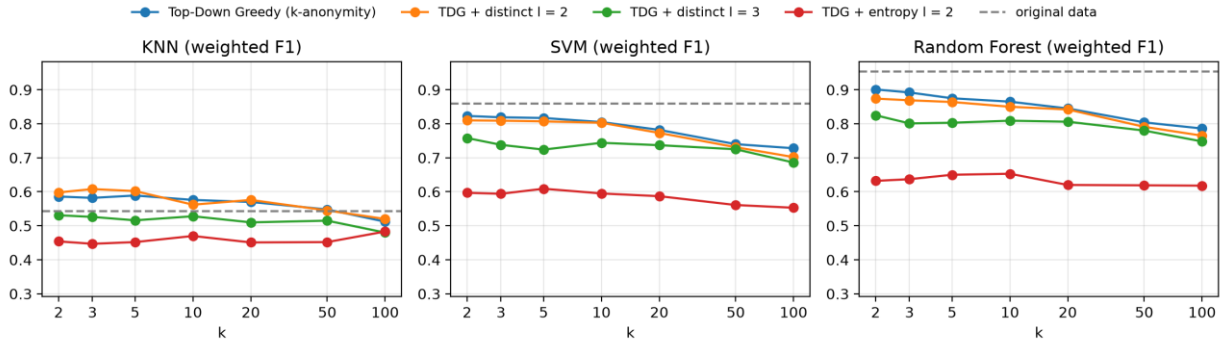


Figure 3: Experiment B. Weighted F1 for predicting ocean_proximity; dashed = original data.

On the original data the F1 scores are 0.543 / 0.859 / 0.953 (KNN / SVM / RF), with the original TDG at k = 10 0.576 / 0.805 / 0.865. Distinct l = 2 costs little (0.562 / 0.803 / 0.850 at k = 10, never more than 0.027 below the original TDG at the same k). Distinct l = 3 (0.528 / 0.744 / 0.809) and especially entropy l = 2 (0.470 / 0.595 / 0.653) cost a lot. This is expected here, because the SA is also the class: l-diversity forces every EC to contain several classes, so the generalised QIs carry less information about the label. In other words, on this dataset protecting ocean_proximity and predicting it are directly in conflict.

5 Discussion and conclusion

Adding l-diversity to TDG works: with our repair step every EC is k-anonymous and l-diverse, and the information loss is clearly lower than with rejecting or retrying invalid splits. On cahousing the SA is strongly connected to the QIs, so l-diversity is not cheap: distinct l = 2 roughly doubles the NCP at small k but still beats simply raising k. The more important lesson is that distinct l-diversity alone gives weak protection on such data: with only three SA values and very skewed ECs the majority guess is still right for most districts. Entropy l-diversity fixes this, but then the data loses most of its use for predicting ocean_proximity.

Limitations: one dataset with only three SA values, which is also the target. The repair is greedy and does not look for the best set of records. We did not measure runtime carefully (single runs took about 2 to 15 s on a laptop). For stronger protection against skewness and similarity attacks, t-closeness would be the next step.

References

[1] A. Machanavajjhala, D. Kifer, J. Gehrke, M. Venkitasubramaniam. l-Diversity: Privacy beyond k-Anonymity. ACM TKDD 1(1), 2007.

[2] J. Xu, W. Wang, J. Pei, X. Wang, B. Shi, A. Fu. Utility-Based Anonymization Using Local Recoding. KDD 2006.

[3] L. Sweeney. k-Anonymity: A Model for Protecting Privacy. IJUFKS 10(5), 2002.

[4] M.-K. Pham. kaylode/k-anonymity toolbox, https://github.com/kaylode/k-anonymity (TDG from https://github.com/fhstp/k-AnonML).

[5] D. Slijepcevic et al. k-Anonymity in Practice: How Generalisation and Suppression Affect Machine Learning Classifiers. Computers & Security, 2021.